No additional pattern classifications were introduced during this review.

Result: The inbound-email candidate review supports two source-confirmed GRASP instances and one unresolved Turbo framework candidate. The Observer question remains shared with geocoding rather than becoming a new independent count.

Now move to the next source area: trip creation, membership and permission handling.
As before, do not begin by assigning pattern names. Trace the actual collaboration first.
Inspect these files and any directly related callers needed to complete the flow:

1. `app/controllers/trips_controller.rb`
2. `app/models/trip.rb`
3. `app/models/trip_member.rb`
4. `app/models/traveler.rb`
5. `app/controllers/trip_members_controller.rb`
6. `app/models/account.rb`
7. `app/models/user.rb`
8. Relevant routes, views or supporting models where needed.

I want to understand three things separately:

1. Trip creation

Trace what happens from the request to create a trip through persistence, owner membership creation and any follow-up work such as cover-image processing.

2. Membership management

Trace how travelers become members of a trip, how membership roles are represented and how members are added, updated or removed.

3. Permission handling

Trace how the application decides whether a user can view or modify a trip.

Do not assume that role values such as owner, planner or viewer constitute Strategy, State or any other pattern.
For the investigation:

1. Identify the actual classes and methods participating in each flow.
2. Give the important source paths and reliable line ranges.
3. Explain where responsibility for creating the trip and its initial membership is placed.
4. Explain how `Trip`, `TripMember`, `Traveler`, `Account` and `User` relate during these operations.
5. Trace the actual permission checks rather than describing generic Rails authorization.
6. Separate application-written behavior from Active Record, controller and routing behavior supplied by Rails.
7. Check whether responsibility is concentrated in one object or coordinated across several objects.
8. Identify any repeated permission or membership logic that appears across controllers.

Only after tracing all three flows, identify design-pattern candidates that are genuinely worth investigating.
For each candidate:

1. Give the possible pattern name and category.
2. Identify the actual collaboration suggesting it.
3. Explain why it might fit.
4. Explain the strongest reason it might be a false positive.
5. State whether it is application-owned, framework-provided or mixed.
6. Leave it UNCONFIRMED at this stage.

Do not force classifications such as Strategy, State, Factory, Composite, Proxy or GRASP patterns unless their defining responsibility or collaboration is supported by the source.
At the end give me:

1. Trip-creation collaboration trace.
2. Membership-management collaboration trace.
3. Permission-check collaboration trace.
4. Participant and source-evidence table.
5. Application versus Rails responsibility split.

6. Candidate pattern classifications, if any.
7. Strongest non-pattern explanation.
8. Anything that needs further checking before candidate verification.

Worked for 2m 10s

DaPlanStan — Trip Creation, Membership and Permission Investigation

Patterns unconfirmed Source Investigation 03 · Original uploaded ZIP

1. Initial source findings

The implementation separates responsibilities across Rails controllers and Active Record models. Three important facts emerge from the code:

1. **Trip creation and owner-membership creation are coordinated by `TripsController#create`, not by a single `Trip` model method.** The controller saves the trip, creates its first `TripMember`, and enqueues cover-image retrieval.
2. **Membership roles are stored as values on `TripMember`.** No application-defined interchangeable role objects or role-specific classes were identified in this source area.
3. **Permissions are enforced differently across controllers.** Some actions require owner or planner membership, others require only that the user belongs to the trip.

The last distinction matters: DaPlanStan does not implement one uniform, centralized authorization policy.

2. Trip creation — complete collaboration trace

Trip creation control flow